



1

三連続 (Three Consecutive)

解説作成者：米山瑛士

「ある 0 以上 $N-2$ 未満の整数 i が存在して、すべての 0 以上 3 未満の整数に対して、 $S[i+j]$ が 'o' であるか判定せよ」という問題です。よって、サンプルプログラム (C++) のように実装すれば良いです。

よくある間違いの例

- N と S をこの順に入力していない。
- i の範囲を間違えている。例えば、 0 以上 N 未満にしている。



2

ダンス (Dance)

Author: 萩原 千晴 (Mendako)

小課題 1

$N = 1$ であるので、クラスには 2 人の生徒がいる。よって、この 2 人の身長之差が D 以下かどうかを判定すれば良い。

```
1  if(abs(A[0] - A[1]) <= D) cout << "Yes" << endl;  
2  else cout << "No" << endl;
```

小課題 2

$D = 0$ であるので、身長が同じ 2 人でしか 2 人組を作れない。よって、すべての身長について、その身長に一致する生徒の人数が偶数であるかどうかを判定すれば良い。

```
1  vector<int> heights(301, 0);  
2  for(int i = 0; i < N; i++) {  
3      heights[A[i]]++;  
4  }  
5  bool ans = true;  
6  for(int i = 1; i <= 300; i++) {  
7      if(heights[i] % 2 == 1) ans = false;  
8  }  
9  if(ans) cout << "Yes" << endl;  
10 else cout << "No" << endl;
```

小課題 3

身長を昇順にソートした後、 $2i - 1$ 番目と $2i$ 番目の身長之差が、すべての $i (1 \leq i \leq N)$ について D 以下になっていれば良い。



日本情報オリンピック 第4回女性部門 (JOIG 2023/2024) 本選
2024 年 1 月 21 日 (オンライン開催)

```
1      sort(A.begin(), A.end());
2      bool ans = true;
3      for(int i = 0; i < N; i++) {
4          if(A[2 * i] - A[2 * i - 1] > D) ans = false;
5      }
6      if(ans) cout << "Yes" << endl;
7      else cout << "No" << endl;
```



3

座席 2 (Seats 2)

小課題 1 ($N \leq 1000$)

問題文の通り、各選手 i について、以下のような処理を行えばよいです。

選手 i についての答えを持つ変数 x を用意する。(初期値は十分大きい値とする)
すべての選手 j について、 $C_i \neq C_j$ ならば x を $\min(x, |X_i - X_j|)$ で置き換える
 x を出力する。

計算量は $O(N^2)$ となり、 $N \leq 1000$ では実行時間制限に間に合います。

小課題 2 ($C_i \leq 10$)

重要な考察として、選手 i に対する答えは、以下の2つのうち小さい方であるというものがあります。

- (1) $X_i -$ (選手 i より座標が小さいか同じ選手のうち選手 i と出身国の違う選手の座標の最大値)
- (2) (選手 i より座標が大きいのか同じ選手のうち選手 i と出身国の違う選手の座標の最小値) $- X_i$

これらは左右対称なので (1) の求め方だけ考えます。((1) の求め方が分かれば (2) は同様に求められます)
この小課題では国の数が少ないことに注目しましょう。選手を座標の小さい順にソートし、前から順にみていきます。その際 $m[j]$ を、「今までに見た国 j 出身の選手の座標の最大値」とします。

すると、選手 i を見た時、「(選手 i より座標が小さいか同じ選手のうち選手 i と出身国の違う選手の座標の最大値)」は、 C_i 以外のすべての j についての $m[j]$ の最大値となります。

この小課題では、 m の長さは 10 しかないので、これは高速に求まります。

一方で、選手 i を見終わった時点で $m[C_i]$ に X_i を入れれば、 m は正しく更新されていきます。(選手は座標でソートされているので、選手 i は今まで見た選手の中で座標が最大です)

これにより、計算量 $O(N \log(N) + N \max C_i)$ で問題が解けます。

小課題 3 (追加の制約なし)

小課題 2 の解説の冒頭にある考察をここでも使い、小課題 2 と同様に選手を座標でソートしたあと前から順に見ていきます。

ここで、常に



日本情報オリンピック 第4回女性部門 (JOIG 2023/2024) 本選
2024 年 1 月 21 日 (オンライン開催)

(a) 今までに見た選手の座標の最大値 m_1 とその最大値をとる人の出身国 c_1

(b) 今までに見た選手のうち、(a) とは出身国の違う選手の座標の最大値 m_2

が分かれば、(選手 i より座標が小さいか同じ選手のうち選手 i と出身国の違う選手の座標の最大値) は求められます。($C_i \neq c_1$ なら m_1 , そうでなければ m_2 となる)

選手 i を見終わった際、 m_1, c_1, m_2 は以下のようにすれはうまく更新できます。

- $C_i = c_1$ なら、 m_1 に X_i を入れる
- $C_i \neq c_1$ なら、 (m_1, c_1, m_2) を (X_i, C_i, m_1) で置き換える (選手 i が (a) になり、もともと (a) だった人が (b) になる)

よって、この問題は計算量 $O(N \log(N))$ で解け、 $N \leq 300\,000$ の制約のもとでは実行時間制限に間に合います。



4

たくさんの数字 (Many Digits)

Author : 蜂矢 倫久 (Mitsubachi)

小課題 1

葵さんが書く整数は1個で、その値は $A_1 + B_1$ である。したがって、この数字の桁数を求めればよい。

これは C++ ならば `to_string` 関数で整数を文字列に変換したのち、その文字列の長さを取得すればよい。
 $O(1)$ で答えが求まる。

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main(){
6      int n;
7      cin >> n;
8      vector<long long> a(n), b(n);
9      for(int i = 0; i < n; i++){
10         cin >> a[i];
11     }
12     for(int i = 0; i < n; i++){
13         cin >> b[i];
14     }
15
16     cout << to_string(a[0] + b[0]).size() << endl;
17 }
```



小課題 2

葵さんが書く N^2 個の整数は for 文を用いることですべて列挙できる。それらすべてに対して小課題 1 の手法で桁数を求めればよい。 $O(N^2)$ で答えが求まる。

```
1      int ans = 0;
2      for(int i = 0; i < n; i++){
3          for(int j = 0; j < n; j++){
4              ans += to_string(a[i] + b[j]).size();
5          }
6      }
7      cout << ans << endl;
```

小課題 3

$A_i = x$ となる i の個数が a 個で、 $B_j = y$ となる j の個数が b 個とする。このとき、 $A_i = x$, $B_j = y$ となる (i, j) の個数は ab 個である。

$1 \leq x \leq 2000$, $1 \leq y \leq 2000$ について $x+y$ の桁数に ab をかけた値を答えに足すという操作を行えばよい。あらかじめ x , y について a , b を求めておくと高速に答えが求まる。

$V = 2000$ として $O(V^2)$ で答えが求まる。

```
1      vector<long long> count_a(2010, 0), count_b(2010, 0);
2      for(int i = 0; i < n; i++){
3          count_a[a[i]]++;
4          count_b[b[i]]++;
5      }
6
7      long long ans = 0;
8      for(int i = 0; i < 2010; i++){
9          for(int j = 0; j < 2010; j++){
10             ans += count_a[i] * count_b[j] * to_string(i + j).size();
11         }
12     }
13     cout << ans << endl;
```



小課題 4

葵さんが書く整数は9桁か10桁であり、特に10桁となるのは $A_i = 500\,000\,000$, $B_j = 500\,000\,000$ のときのみである。

よって、 $A_i = 500\,000\,000$ となる i の個数が a 個で、 $B_j = 500\,000\,000$ となる j の個数が b 個とすると答えは $9N^2 + ab$ である。 $O(N)$ で答えが求まる。

```
1      long long count_a = 0, count_b = 0;
2      for(int i = 0; i < n; i++){
3          if(a[i] == 500000000){
4              count_a++;
5          }
6          if(b[i] == 500000000){
7              count_b++;
8          }
9      }
10
11     cout << (long long)(n) * n * 9 + count_a * count_b << endl;
```

小課題 5

葵さんが書く整数は9桁か10桁である。 i ごとに $A_i + B_j$ が10桁になる j の個数を求める。

このような j は $10^9 - A_i \leq B_j$ を満たす。あらかじめ B_j を保存する配列を sort しておくと lower_bound により $O(\log N)$ で j の個数を求めることができる。

$O(N \log N)$ で答えが求まる。

```
1      sort(b.begin(), b.end());
2
3      long long ans = (long long)(n) * n * 9;
4      for(int i = 0; i < n; i++){
5          ans += b.end() - lower_bound(b.begin(), b.end(), 1000000000 - a[i]);
6      }
7      cout << ans << endl;
```




小課題 6

$A_i + 1$ から $A_i + N$ までの桁数の合計を高速に求めることを考える。

これはあらかじめ 1 から x までの桁数の合計を $1 \leq x \leq 300\,000$ について求める。これは累積和を用いれば高速に求まる。 $A_i + 1$ から $A_i + N$ までの桁数の合計は 1 から $A_i + N$ までの桁数の合計から 1 から A_i までの桁数の合計を引いた値であるので $O(1)$ で求まる。

$V = 150\,000$ として $O(V + N)$ で答えが求まる。

```
1      vector<long long> sum(150100 + n, 0);
2      for(int i = 1; i < sum.size(); i++){
3          sum[i] = sum[i - 1] + to_string(i).size();
4      }
5
6      long long ans = 0;
7      for(int i = 0; i < n; i++){
8          ans += sum[a[i] + n] - sum[a[i]];
9      }
10
11     cout << ans << endl;
```

小課題 7

小課題 6 と同様に $A_i + 1$ から $A_i + N$ までの桁数の合計を高速に求めることを考える。

$A_i + 1$ から $A_i + N$ までのうち d 桁であるような数の個数を $1 \leq d \leq 10$ について求めることを考える。このような個数は l を 10^{d-1} と $A_i + 1$ のうち小さくない方, r を $10^d - 1$ と $A_i + N$ のうち大きくない方として, $r - l + 1$ と 0 のうち大きくない方である。

よって, $O(N)$ で答えが求まる。

```
1      long long ans = 0;
2      for(int i = 0; i < n; i++){
3          long long left = 1, right = 10;
4          for(int d = 1; d <= 11; d++){
5              long long l = max(left, a[i] + 1), r = min(right - 1, a[i] + n);
6              ans += d * max(0LL, r - l + 1);
7          }
8      }
```



```
7         left *= 10;
8         right *= 10;
9     }
10 }
11
12 cout << ans << endl;
```

満点

$1 \leq d \leq 10$ について i を固定したときに $A_i + B_j$ が d 桁となるような j の個数を求めることを考える.

このような j は $10^{d-1} - A_i \leq B_j \leq 10^d - 1 - A_i$ を満たす. したがって, j の個数は小課題 5 のように, あらかじめ B_j を保存する配列を sort しておくと lower_bound により $O(\log N)$ で j の個数を求めることができる.

よって, $O(N \log N)$ で答えが求まる.

```
1     sort(b.begin(), b.end());
2
3     long long ans = 0;
4     long long x = 1;
5     for(int d = 1; d <= 10; d++){
6         // d 桁の数字の個数を数える
7         for(int i = 0; i < n; i++){
8             // a[i] + b[j] が d 桁となるような j の数を数える
9             // 10^{d-1} <= a[i] + b[j] < 10^d
10            // 10^{d-1} - a[i] <= b[j] < 10^d - a[i]
11            long long left = x - a[i], right = 10 * x - a[i];
12            auto itr1 = lower_bound(b.begin(), b.end(), right);
13            auto itr2 = lower_bound(b.begin(), b.end(), left);
14            ans += d * (itr1 - itr2);
15        }
16        x *= 10;
17    }
18
19    cout << ans << endl;
```



5

名前 (Name)

Author: 米田 寛峻 (square1001)

0 はじめに

この問題では、 S と T を部分列に含み、どの同じ文字の間にも別の文字が K 文字以上あるような最短の文字列の文字数を求める必要があります。

1 小課題 1 ($S = T, K = 0$)

$S = T, K = 0$ では、文字列 S がそのまま条件を満たします。その文字数 N を出力すればよいです。

2 小課題 2 ($S = T, K = 1$)

$S = T, K = 1$ では、文字列 S で同じ文字が隣接する箇所があった場合、そのままでは条件を満たしません。同じ文字が隣接する箇所について、間に別の文字を 1 文字挿入してやれば、条件を満たすようになります (そしてそれは当然最短です)。文字数は $N + (S[i] = S[i + 1])$ となる i の個数) となります。

3 小課題 3 ($S = T$)

小課題 2 と同じように考えます。同じ文字の間に $K - 1$ 文字未満しかない箇所があった場合、いくつか文字を挿入して条件を満たすようにしてやる必要があります。

具体例として、 $K = 3$ で S, T が `abcdbcda` の場合を考えます。まず、2 つの `b` の間には 2 文字しかないので、その間に別の文字を 1 文字挿入する必要があります。`bc` の間、`cd` の間、`db` の間のどこに挿入するのが良いでしょうか？答えは、一番後ろの `db` の間に挿入するのが最適です。理由は、そうすれば 2 つの `c` の間と 2 つの `d` の間 (両方とも間に 2 文字しかないので) にも 1 文字挿入したことになり^{*1}、より条件の達成に近づくからです (図 1)。

^{*1} 実際には 2 つの `a` の間にも挿入されていますが、2 つの `a` の間には既に 3 文字以上あるので考える必要はありません。

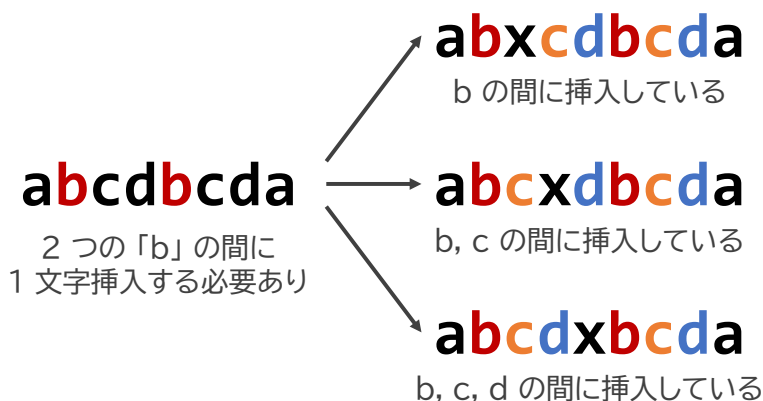


図1 abcbcbda の2つのbの間の挿入位置による文字列の変化

しかし、この考え方ではどこに挿入するか定まらない場合もあります。例えば $K = 3$ で文字列が **xayxzyzbzab** のときに、2つの **y** の間に1文字挿入することを考えます。このとき

- **yx** の間に何か別の文字を挿入すると、2つの **y** の間、2つの **x** の間に挿入したことになる
- **zy** の間に何か別の文字を挿入すると、2つの **y** の間、2つの **z** の間に挿入したことになる

となり、どちらを選べばよいかの判断がつきません^{*2}。

しかし、 $S[l] = S[r]$ ($1 \leq r-l \leq K-1$) となる箇所の中で r が最も左にあるものに着目すれば、必ず「挿入する場所として一番右を選べばよい」と決まります^{*3}。前の例ならば、**y** ではなく **x** に着目すべきだったということです。「このような (l, r) を見つけ、 $S[r-1]$ と $S[r]$ の間に（左右 K 文字とは異なる）1文字を挿入する」を条件を満たす文字列ができるまで繰り返すことで、最適解が求まります。 (l, r) を探すのに計算量 $O(NK)$ かかり、最大で NK 回程度の繰り返しが起こるので、全体の計算量は $O(N^2K^2)$ となります。

もっとシンプルな計算量 $O(NK)$ の解法もあります。

1. 答えの文字列を Z とする。最初、 Z を空文字列とする。
2. $i = 1, 2, \dots, N$ の順に以下を繰り返す。
 - Z の末尾にダミーの文字を ($S[i]$ が直前の同じ文字と K 文字以上空けるように) 必要最小限な数だけ追加した後、 Z の末尾に $S[i]$ を追加する。

このように、現時点で最も良いと考えられる選択を繰り返して答えを出す手法を **貪欲法** といいます。

^{*2} 最適解となるのは前者の場合のみとなります。条件を満たす最適な文字列の一例は **xayPxzybQzab** です。

^{*3} 理由は少し難しいですが、ある（間に $K-1$ 文字以下しかない）2つの文字の間に挿入される条件が「一定より右の場所に挿入すること」となるからです。



4 小課題 4 ($K = 0$)

$K = 0$ の場合は「 S と T を部分列に含む最短の文字列」の文字数を求めることになります。この問題は、動的計画法^{*4}という手法を用いて解くことができます。

まずは、解法をイメージしやすいように、図2のように S と T を同じ列に同じ文字が来るように縦に並べるときに最短何列必要か、という問題として考えます。図2は、 S が JOIIIOI で T が JOIGEGOI のときに、最短の文字列 JOIGEIGOI が作られる場合の例を示しています。

S:	J	O	I			I		O	I
T:	J	O	I	G	E		G	O	I

図2 JOIIIOI と JOIGEGOI を部分列に含む最短の文字列

この並べ方を左から順に作っていくことを考えます。状態 (i, j) を「現時点で S の i 文字目まで、 T の j 文字目までが作られた状態」とします。例えば、図2で6列目まで作ったら「状態 $(4, 5)$ 」です (図3)。

S:	J	O	I			I	
T:	J	O	I	G	E		

状態 $(4, 5)$

図3 状態 $(4, 5)$: S の4文字目まで、 T の5文字目までが作られた状態

$d[i][j]$ を「状態 (i, j) にするために最小何列必要か」とします (例 : $d[4][5] = 6$) とします^{*5}。これを (i, j)

^{*4} 動的計画法 (Dynamic Programming, DP) とは、小さいスケールの問題から順に、これまでの結果を利用して答えを積み上げていくことで、最終的な問題の答えを求める方法のことです。動的計画法を使って解く問題は、情報オリンピックでもよく出題されます。分からない方はインターネットなどで調べてみましょう。書籍による説明では、「競技プログラミングの鉄則」(米田優峻著) p.105-152 などがあります。

^{*5} $d[i][j]$ を「『 S の i 文字目まで』と『 T の j 文字目まで』を部分列に含む最短の文字列の文字数」としても結果は同じですが、図2のような形で考えたほうが DP の式が立てやすいです。



の小さい順に計算していくことを考えます。まず、状態 (i, j) にたどり着くためには、直前の状態から以下のいずれかのようにする必要があります。

- 状態 $(i-1, j)$ から、次の列で 1 行目に $S[i]$ を書く。
- 状態 $(i, j-1)$ から、次の列で 2 行目に $T[j]$ を書く。
- 状態 $(i-1, j-1)$ から、次の列で 1 行目に $S[i]$ 、2 行目に $T[j]$ を書く。これは $S[i] = T[j]$ の場合のみ行える。

よって、 $d[i][j]$ は以下の式で求めることができます。

$$d[i][j] = \begin{cases} \min \{d[i-1][j] + 1, d[i][j-1] + 1\} & (S[i] \neq T[j]) \\ \min \{d[i-1][j] + 1, d[i][j-1] + 1, d[i-1][j-1] + 1\} & (S[i] = T[j]) \end{cases} \quad (1)$$

この問題の答えは $d[N][M]$ ですので、計算量 $O(NM)$ で答えを求めることができます。

小課題 4 には別の解法もあります。最長共通部分列問題 (LCS) というよく知られた問題があります。これは、文字列 S と文字列 T の両方の部分列として現れる最長の文字列を求めてください、というものです (例: JOIIOI と JOIGEOI の最長共通部分列は JOIOI)。このとき、元の問題の答えは、 $N + M - (S$ と T の最長共通部分列の長さ) となります。最長共通部分列は動的計画法を使って計算量 $O(NM)$ で求められるので、元の問題も計算量 $O(NM)$ で解けました。

5 小課題 5 ($K = 1, N \leq 25, M \leq 25$)

基本的には小課題 4 の解法と同じように考えたいです。でも、 $K = 1$ の場合は、隣接する文字が同じではいけないという制約があります。なので、図 2 と同様に文字を並べると、どうしても $S[i]$ も $T[j]$ も書かない列が出てくる場合があることに注意してください (図 4)。

S:	C	O	M		M	I	T			T	E				E
T:							T	E	R			R	A	C	E
	C	O	M	X	M	I	T	E	R	T	E	R	A	C	E

図 4 $K = 1$ 、 S が COMMITTEE、 T が TERRACE のときの最短の文字列

$d[i][j][k]$ を「状態 (i, j) で、最後の列の文字が k (アルファベット 52 種類のいずれか) にするために、最小何列必要か」とします。 (i, j, k) の時点から、次の列を作ったとき、以下のいずれかになります。



1. 次の列に何も書かないことで、 (i, j, k') に遷移する (ただし $k \neq k'$)。
2. 次の列で 1 行目に $S[i+1]$ を書くことで、 $(i+1, j, S[i+1])$ に遷移する。
3. 次の列で 2 行目に $T[j+1]$ を書くことで、 $(i, j+1, T[j+1])$ に遷移する。
4. 次の列に 1 行目に $S[i+1]$ 、2 行目に $T[j+1]$ を書くことで、 $(i+1, j+1, S[i+1])$ に遷移する。これは $S[i+1] = T[j+1]$ の場合にのみ行える。

よって、各 (i, j, k) を頂点としたグラフで、上に述べた状態遷移に従って (辺の重みが 1 の) 有向辺を張ったものを考えたときに、頂点 $(0, 0, ?)$ から頂点 $(N, M, ?)$ までの最短距離が答えになります。これは、文字の種類数を $\sigma = 52$ として、 $NM\sigma$ 頂点 $NM\sigma^2$ 辺程度のグラフになります。よって、幅優先探索 (BFS) をすれば^{*6}、答えが計算量 $O(NM\sigma^2)$ で求まります^{*7}。

6 小課題 6 ($K = 1$)

小課題 5 の解法は、直前の文字 k を限定することによって、高速化することができます。

現時点の状態が (i, j) であるとき、最後の列の文字 k として「 $S[i]$ 」「 $T[j]$ 」「ダミーの文字 (列に何も書かない場合)」の 3 通りしか考えなくてよいです。すると、グラフの頂点数を $3NM$ 程度に、辺数を $12NM$ 程度に減らせます^{*8}。よって、幅優先探索をすれば、答えが計算量 $O(NM)$ で求まります。

小課題 6 には、動的計画法 (DP) を使った別の解法もあります。今度は、図 5 のように、何も書かない列を許さないことを考えます。すると、同じ文字が隣接する場合があります、その時は「何かダミーの文字を入れる」ことになります。

S:	C	O	M	M	I	T			T	E				E
T:						T	E	R			R	A	C	E

↑
何かダミーの文字を
入れる必要がある

図 5 $K = 1$ 、 S が COMMITTEE、 T が TERRACE のときの最短の文字列 (空列を許さない場合)

$d[i][j][0]$ を「状態 (i, j) で最後の列の文字が $S[i]$ 」、 $d[i][j][1]$ を「状態 (i, j) で最後の列の文字が $T[j]$ 」に

^{*6} この方法では、動的計画法を直接使えないことに注意してください。理由は、作られるグラフにループが生じるからです。

^{*7} 頂点 $(0, 0, k)$ からの最短経路を各 k に対して求めれば計算量 $O(NM\sigma^3)$ かかりますが、幅優先探索の最初の時点でキューにすべての $(0, 0, k)$ を追加してやれば、1 回しか幅優先探索を行う必要がありません。

^{*8} 各 (i, j) に対して、小課題 5 の解法の説明にある 1, 2, 3, 4 の遷移は最大でも 9, 1, 1, 1 通りとなります。



するために最小何列必要か、とします。小課題 4 と同じように考えて動的計画法の式を立てます。まずは $d[i][j][0]$ に関して以下の式が作れます*9。

$$d[i][j][0] = \begin{cases} \min \{d[i-1][j][0] + c(S[i-1], S[i]), d[i-1][j][0] + c(T[j], S[i])\} & (S[i] \neq T[j]) \\ \min \{d[i-1][j][0] + c(S[i-1], S[i]), d[i][j-1][0] + c(T[j], S[i]), \\ d[i-1][j-1][0] + c(S[i-1], S[i]), d[i-1][j-1][1] + c(T[j-1], S[i])\} & (S[i] = T[j]) \end{cases} \quad (2)$$

ただし、 $c(x, y)$ は「 $x = y$ のとき 2、 $x \neq y$ のとき 1」であり、 x と y が隣接したときに追加で必要になる文字数を表します。 $d[i][j][1]$ に関しても同様の式が作れます。よって、答えが計算量 $O(NM)$ で求まります。

7 小課題 7, 8

小課題 7, 8 は、前の小課題よりも難易度が高いので、解説も少し難しい内容になります。

小課題 5 の解法をそのまま $K \geq 2$ の場合に一般化しようとする、最後の K 列の文字を状態として持つておく必要がある、文字の種類数を $\sigma = 52$ として、グラフの頂点数が $O(NM\sigma^K)$ となり、計算量が $O(NM\sigma^{K+1})$ となってしまいます。これでは実行時間制限には到底間に合わない、なんとかして「直前 K 列の情報」の量を削減できないか考えます。

現在状態 (i, j) まで作ったとします。このとき、最後の列は「1 行目に $S[i]$ を書く」「2 行目に $T[j]$ を書く」「 $S[i] = T[j]$ の場合に、1 行目に $S[i]$ を書き、2 行目に $T[j]$ を書く」「何も書かない」の 4 通りがあり得ます（このとき最後の列の文字はそれぞれ「 $S[i]$ 」「 $T[j]$ 」「 $S[i] = T[j]$ 」「ダミーの文字」となります）。

では、その前の列まで考えると何通りあるのでしょうか？各「最後の列のパターン」ごとに 4 が通り考えられるので、 $4 \times 4 = 16$ 通り考えられます（図 6）。

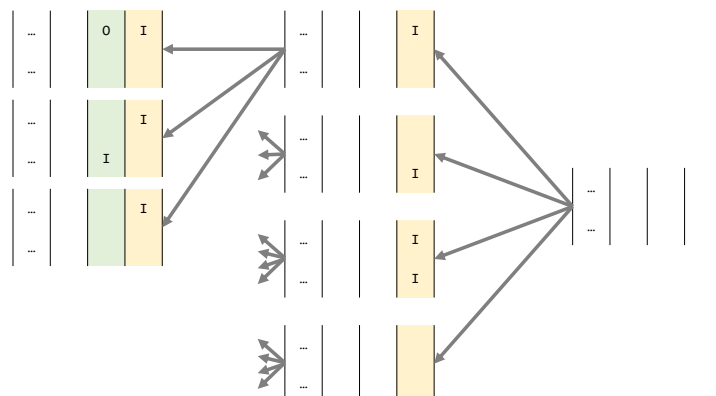


図 6 S が JOIIOI、 T が JOIGEOI のときの最後の列、最後から 2 番目の列の可能性

*9 $d[i][j][0]$ では最後の列の文字が $S[i]$ なので、直前の状態は $(i-1, j)$ か $(i-1, j-1)$ である必要があります。



このように考えていくと、最後の K 列の状態は多くとも 4^K 通りになります。これは元々 σ^K 通りだったの比べると大きな改善です。そうすると、小課題 5, 6 と同じようなやり方で幅優先探索で解くと、計算量 $O(NM4^K)$ で答えが求まります。

なお、 $S[i] = T[j]$ の場合に「1 行目に $S[i]$ を書く」「2 行目に $T[j]$ を書く」パターンを考えなくても最適解は同じなので、状態数を 3^K 通りに減らして、計算量 $O(NM3^K)$ で解くこともできます。

動的計画法で解くこともできます。グラフからループをなくすために、例えば「『何も書かない』が K 連続するようにはしない」とする方法が考えられます。動的計画法を使うと、先ほどの幅優先探索の解法と同じ計算量オーダーですが、(定数倍の面で) 比較的高速なプログラムを書くことができます^{*10}。

この問題では、制約が $N \leq 500, M \leq 500, K \leq 3$ で実行時間制限 1 秒と厳しめなので、実装のやり方やプログラミング言語によっては小課題 8 ($K \leq 3$) や、場合によっては小課題 7 ($K \leq 2$) でも実行時間制限超過 (TLE) になる可能性があります。しかし、実装の工夫をすれば、Python (PyPy3) でも 0.3 秒以内の実行で満点を取ることができます。

8 補足

日本情報オリンピックのウェブサイト上に、小課題 3, 8 の解法を実装した C++, Python プログラムを載せています。ご参考になれば幸いです。

^{*10} この理由として、キューを使わずに済むこと、計算式がより単純化されることなどが挙げられます。



6

感染シミュレーション (Infection Simulation)

解説作成者：米田優峻

小課題 1 (累計 2 点)

制約： $Q \leq 5$ ，すべての客が時刻 0 に来店し，時刻 10 に退店する。

この小課題では，もし感染力 x が 10 以下であれば全員に感染し，そうでなければ最初に感染していた 1 人しか感染しません。したがって， $x \leq 10$ であれば N を，そうでなければ 1 を出力するプログラムを作成すると，正解を出すことができます。

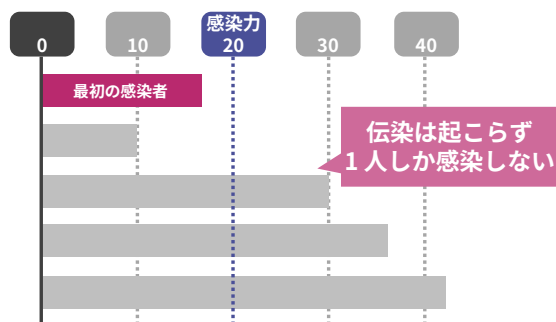
小課題 2 (累計 5 点)

制約： $Q \leq 5$ ，すべての客が時刻 0 に来店する。

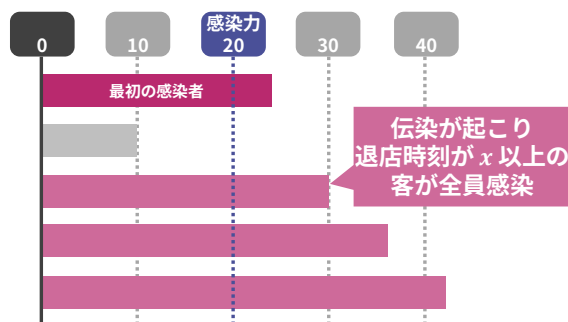
まず，最初に感染した客の退店時刻が感染力 x より小さい場合，伝染が起こる前にその客が帰ってしまいます。そのため，最終的な感染者数は 1 人となります。

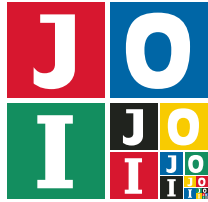
そうでない場合，退店時刻が x かそれ以降である客が全員感染します。そのため，最終的な感染者数は $R_i \geq x$ を満たす i ($1 \leq i \leq N$) の個数となります。単純に for 文を使って個数を計算すると，計算量は $O(NQ)$ となり， $Q \leq 5$ という制約下では実行時間制限に間に合います。

(1) 最初の感染者の退店時刻が 感染力 x より早い



(2) 最初の感染者の退店時刻が 感染力 x かそれ以降





小課題 3 (累計 11 点)

制約：すべての客が時刻 0 に来店する。

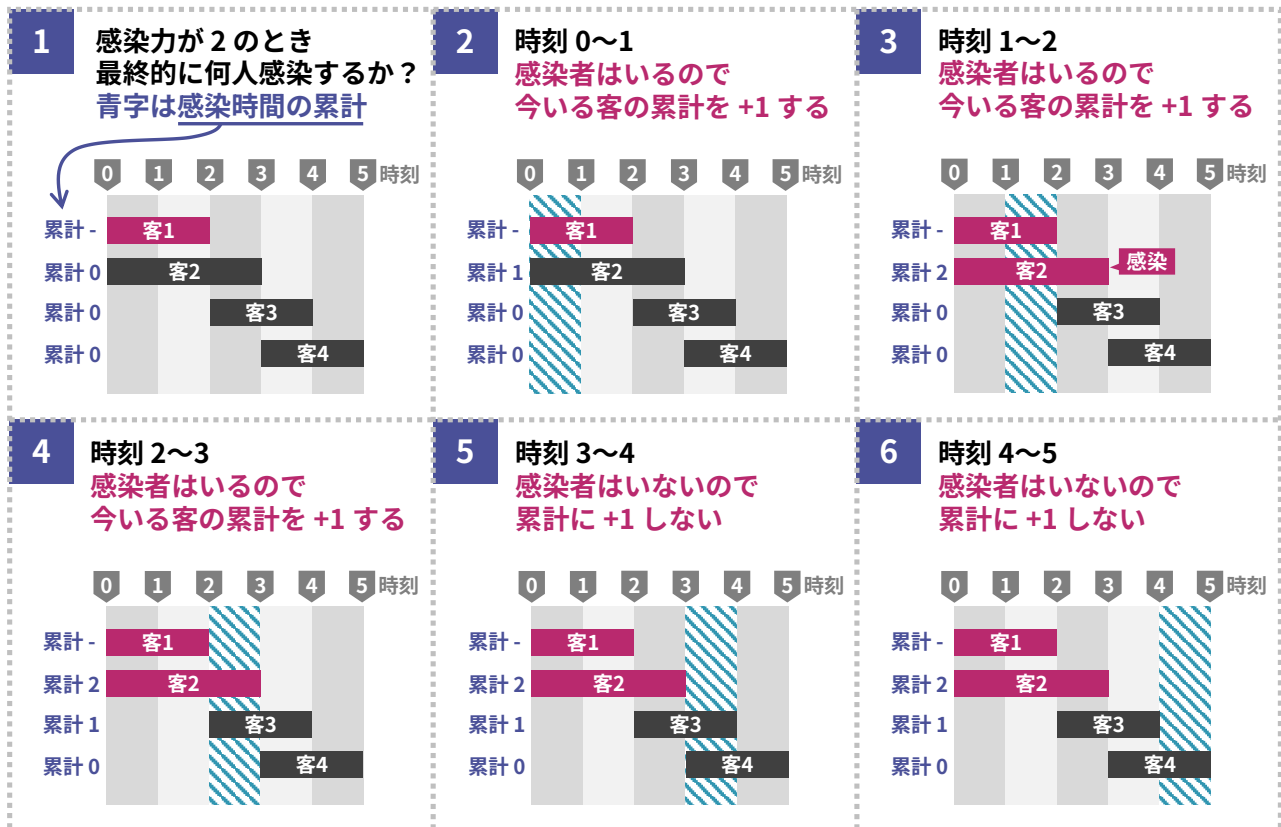
小課題 2 の解法では、 $R_i \geq x$ を満たす i の個数を求める際に単純な for 文を使ったため、計算量が $O(NQ)$ となりました。本小課題の制約は $N, Q \leq 100\,000$ であるため、残念ながらこの解法では TLE となります。

しかし、 R_i をソートして二分探索をすることを考えてみましょう。すると、計算量が $O((N+Q)\log N)$ に削減され、正解を出すことができます。

小課題 4 (累計 21 点)

制約： $N \leq 500$, $Q \leq 5$, 時刻は 500 以下

下図のように、時刻 1 ごとに「感染時間の累計」を更新していくシミュレーションを実装すると、この小課題に正解します。自然に実装した場合、時刻の最大値を T とするとき、計算量は $O(NQT)$ となります。





小課題 5 (累計 32 点)

制約: $N \leq 500$, $Q \leq 5$

この小課題では、時刻の最大値が 10^9 まであり得るため、小課題 4 のような「時刻 1 ごとにシミュレーションする解法」は通用しません。

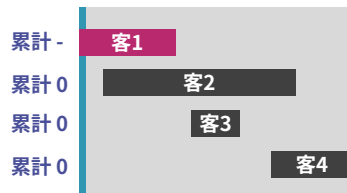
そこで、「来店時刻／退店時刻として出てこない時刻を飛ばしてシミュレーションする」という発想を使うと、計算回数を削減することができます。(このような発想は座標圧縮と呼ばれています)

たとえば客が 4 人おり、それぞれの客が店にいる時刻が 10~40, 20~80, 45~60, 70~95 であったとします。このとき、小課題 4 の解法では時刻 10, 11, 12, 13, ..., 95 のように時刻 1 ごとにシミュレーションを行いました。しかし、時刻 10, 20, 40, 45, 60, 70, 80, 95 のみを考え、間の時刻を飛ばしてシミュレーションするという方法を使うと、計算回数が削減されます。イメージ図を以下に示します。

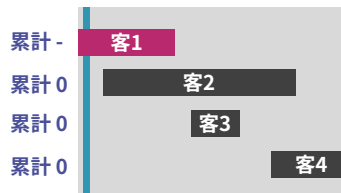
なお、「来店時刻／退店時刻として出てくる時刻」としては $2N$ 個あるため、計算量は $2N \times N \times Q = O(N^2Q)$ となります。

小課題 4 の解法 (感染力 15 の場合)

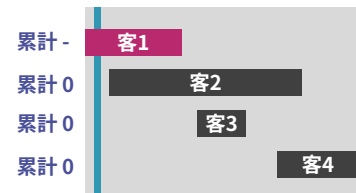
時刻 10~11



時刻 11~12

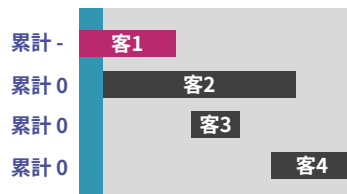


時刻 12~13

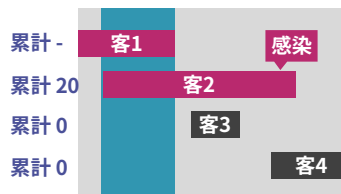


小課題 5 の解法 (感染力 15 の場合)

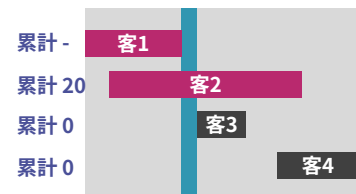
時刻 10~20



時刻 20~40



時刻 40~45





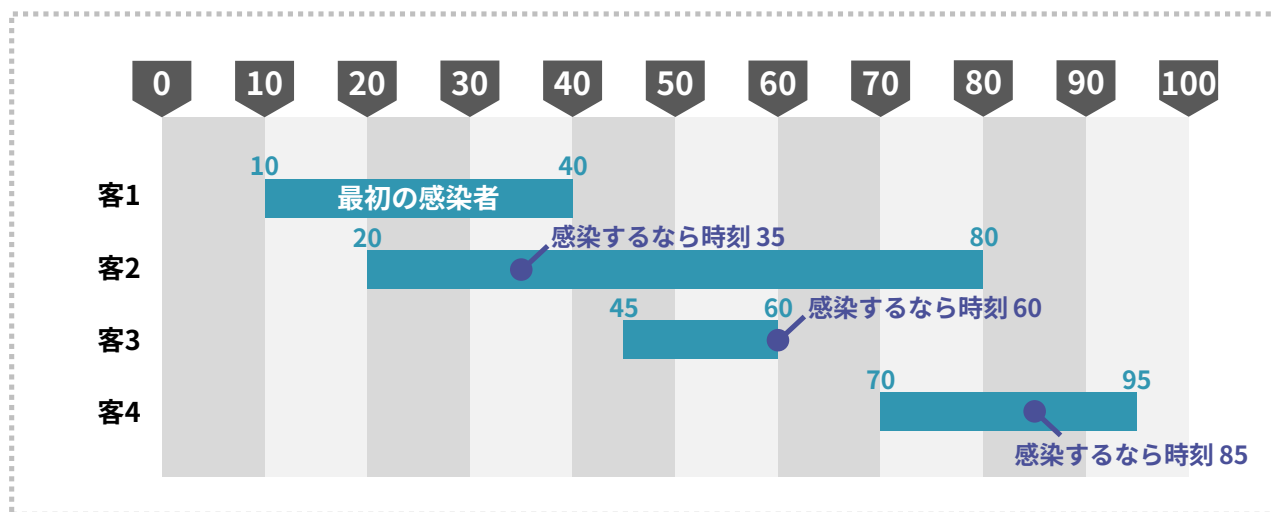
小課題 6 (累計 48 点)

制約: $Q \leq 5$

それでは、先程説明した愚直シミュレーションを何とかして高速化することはできないのでしょうか。まず重要な考察として、一度感染者がいなくなったら、もう二度と感染は起こりません。

そのため、感染力が x であるとき、もし客 i が感染するならば、その感染時刻 I_i は必ず (最初の感染者の来店時刻と L_i の大きい方) $+ x$ となります。たとえば下図の例で感染力が 15 である場合、 $I_2 = 20 + 15 = 35$ 、 $I_3 = 45 + 15 = 60$ 、 $I_4 = 70 + 15 = 85$ となります。

したがって、現在の感染者数を表す変数 `NumInfections` を管理し、来店時刻 L_i 、退店時刻 R_i 、感染時刻 I_i の $3N$ 個の時刻のみを考えたシミュレーションをすると^{*1}、計算量 $O(N \log N)$ で答えを出すことができます。詳しい実装は JOI のホームページに掲載されている実装例を参照してください。



(次ページへ続く)

^{*1} 小課題 5 で、来店時刻 L_i 、退店時刻 R_i の $2N$ 個の時刻のみを考えたシミュレーションを行ったことを思い出しましょう。



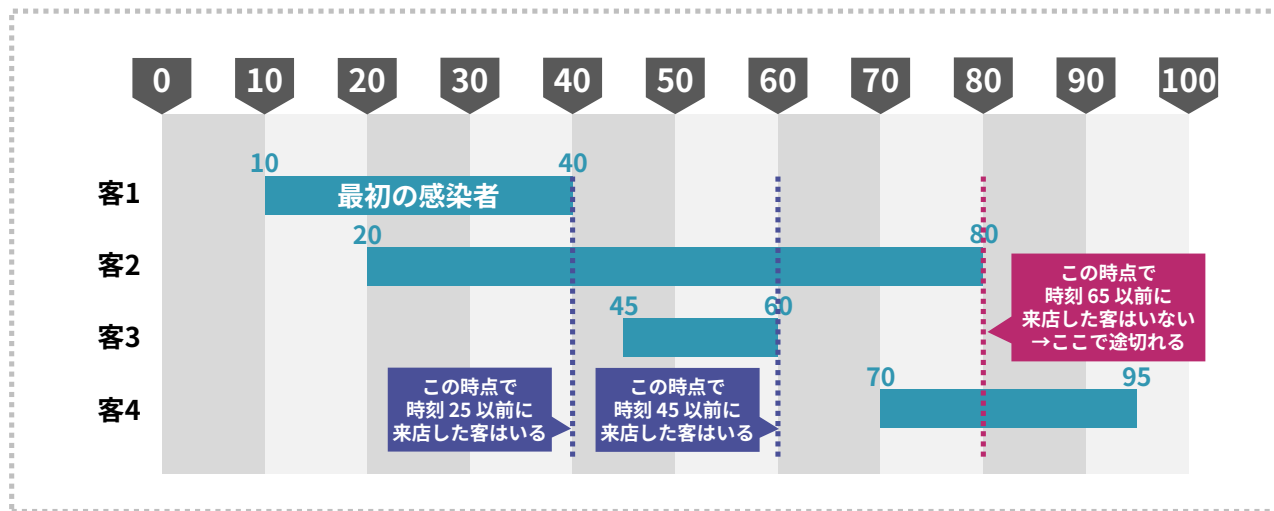
小課題 7, 8 (累計 75 点)

制約: $L_1 \leq L_2 \leq \dots, L_N, R_1 \leq R_2 \leq \dots \leq R_N, P_i = 1$ 。なお、ここでは最初の感染者である客 1 の来店時刻が一番早いという前提で話を進める。

ここまでは、シミュレーションの高速化という方針で解説を行いました。しかし、ここからは少し考え方を変えて、感染者が店から消える時刻に着目してみましょう*²。まず、感染者が店から消える時刻は、以下の条件を満たす時刻 R_j の最小値となります。*³

- 客 j の退店時刻 R_j が終わった時点で、次のような客が店にいない。
- 来店時刻が $R_j - x$ かそれより前である。

例として、入力例 1 (下図) の場合を考えてみましょう。まず時刻 $R_1 = 40$ が終わった時点では、来店時刻が $40 - 15 = 25$ かそれより前の客が店にいるので、時刻 40 では感染者が消えません。続いて時刻 $R_2 = 60$ が終わった時点でも、来店時刻が $60 - 15 = 45$ かそれより前の客が店にいるので、時刻 60 では感染者が消えません。しかし時刻 $R_3 = 80$ が終わった時点では、来店時刻が $80 - 15 = 65$ かそれより前の客が店にいません。よって時刻 80 に店から感染者が消えます。



*² 小課題 6 で、一度感染者が店になかったら、もう二度と感染が起こらないという考察がありました。そのため、感染者が消える時刻が計算できれば、この問題はほぼ解けたも同然です。

*³ もちろん、「最初の感染者の退店時刻かそれより後」の中での最小値を指しています。



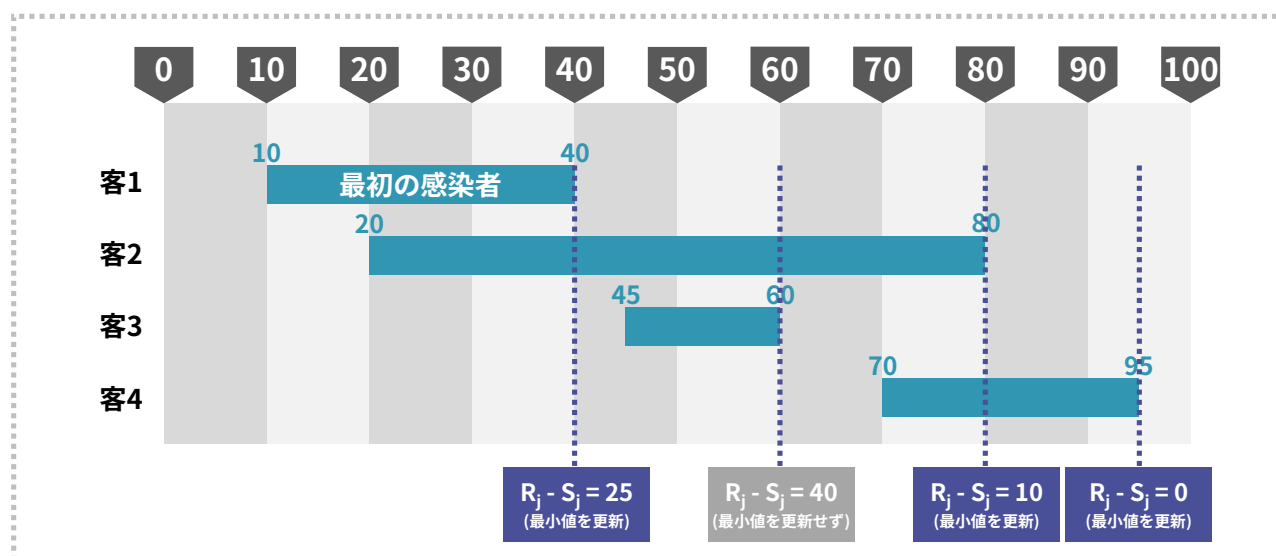
したがって、各 $j = 1, 2, \dots, N$ について、時刻 R_j が終わった時点で店にいる客における来店時刻の最小値を S_j とするとき、感染力が x のシナリオにおける「感染者が消える時刻」は次式で表されます。

- $R_j - S_j < x$ となる最小の R_j

そこで $S_1, S_2, \dots, S_N$ の値は、ソートを利用することで計算量 $O(N \log N)$ で求めることができます。また、 $R_j - S_j < x$ となる最小の x の値についても、

- R_j の小さい順に見ていったときに $R_j - S_j$ の最小値を更新する位置

を前計算しておけば、二分探索によって計算量 $O(\log N)$ で求めることができます (下図参照)。以上で「感染者が消える時刻」を高速に計算することが出来ました。



最後に、本題である「何人感染するか」の計算に戻ります。感染力が x 、感染が止まる時刻を t とするとき、客 i が感染する条件は以下の式で表されます。

- 来店時刻 L_i が $t - x$ かそれ以前
- 店にいた時間 $R_i - L_i$ が x 以上である^{*4}

したがって、感染者数の計算は「二次元平面上の座標 $(L_1, R_1 - L_1), \dots, (L_N, R_N - L_N)$ に点があるとき、ある長方形領域に何個の点があるかを求める」という典型問題と等価です。そしてこの典型問題は平面走査 + セグメント木により $O((N + Q) \log N)$ 時間で解けるため、小課題8に正解することができます。

^{*4} もちろん、最初の感染者である客1の来店時刻が一番早いとは限らない場合、小課題9の解説に書かれている通り、さらに「退店時刻 R_i が (客1の来店時刻) + x かそれ以降」という条件が必要になります。しかし、本小課題では $P_i = 1$ であり、客1が一番早く来る仮定をおいても良いため、この条件を考える必要はありません。



小課題 9, 10 (累計 100 点)

制約：追加の制約はない

まずは「感染者が消える時刻」について考えましょう。小課題 8 と同様に各客 j ($1 \leq j \leq N$) に対して S_j を定義したとき、感染者が消える時刻は次のようになります。

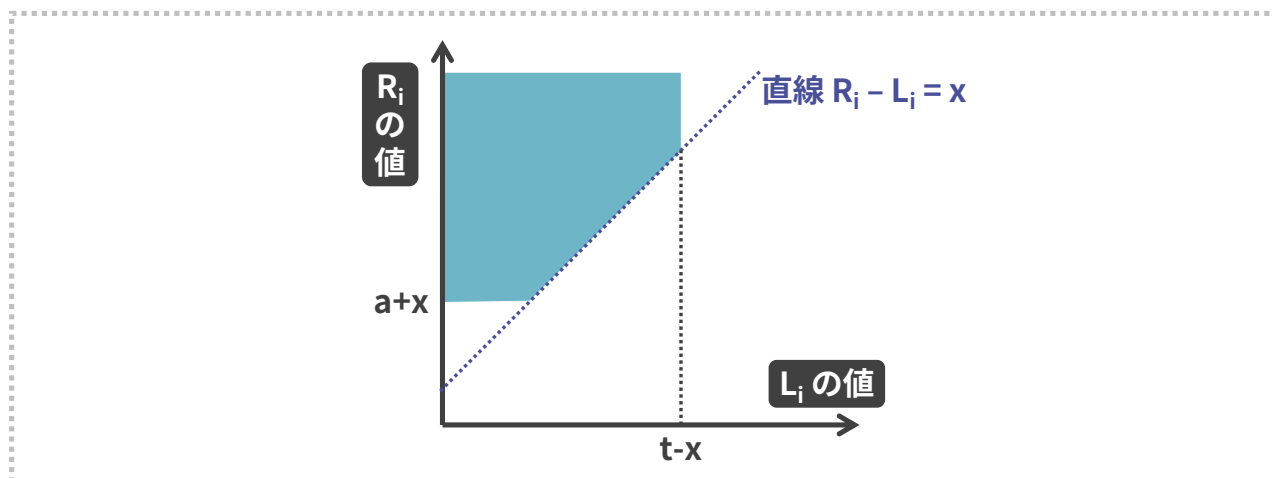
- $R_j \geq$ (最初の感染者の退店時刻) を満たす中における, $R_j - S_j < x$ となる最小の R_j

小課題 8 と比べて「 $R_j \geq$ (最初の感染者の退店時刻)」という条件が新たに追加されているため、小課題 8 のように最小値の更新位置を前計算する方法では解くことができません。しかし、RMQ^{*5} 上で二分探索をすると、各シナリオについて $O(\log^2 N)$ 時間で答えを出すことができます。

問題は何人が感染するかです。感染力が x 、感染が止まる時刻を t 、最初の感染者の来店時刻を a とするとき、客 i が感染する条件は以下の式で表されます。小課題 8 と比べて 2 つ目の条件が増えています。

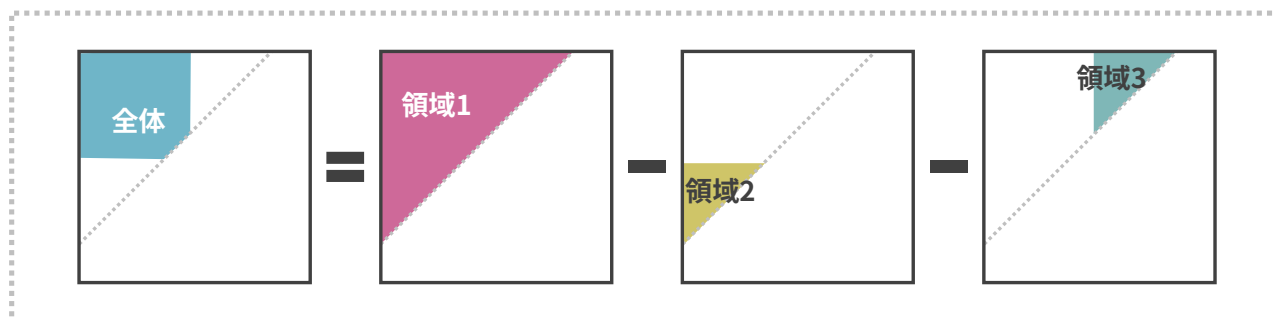
- 来店時刻 L_i が $t - x$ かそれ以前
- 退店時刻 R_i が $a + x$ かそれ以降
- 店にいた時間 $R_i - L_i$ が x 以上である

したがって、二次元平面上に点 $(L_1, R_1), (L_2, R_2), \dots, (L_N, R_N)$ をプロットしたとき、感染者数は下図のような領域に含まれる点の数と同じになります。



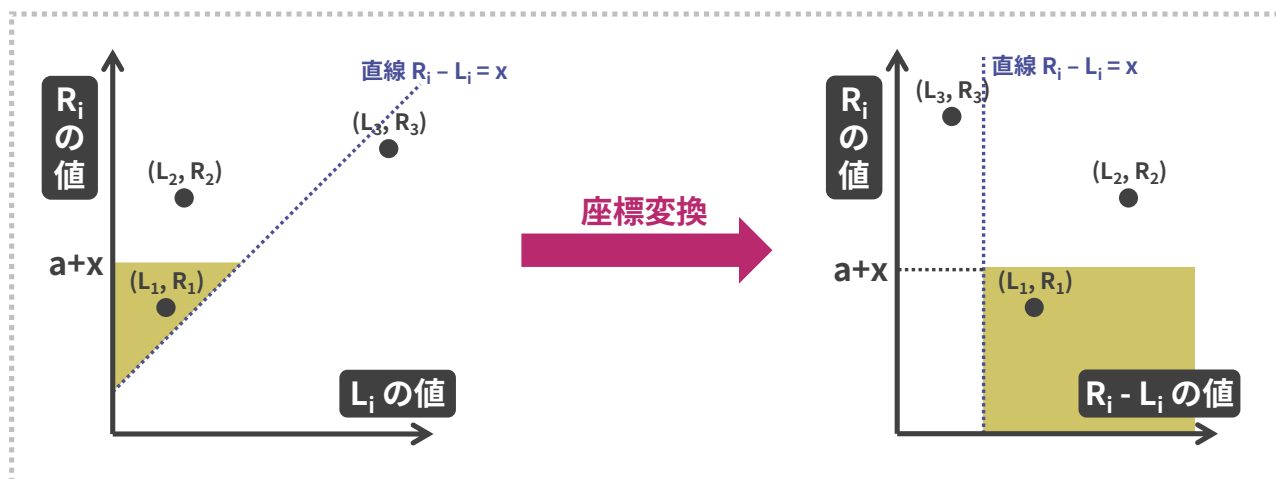
^{*5} Range Minimum Query の略。セグメント木の一種。知らない方は Google 検索を試してみよ。

それでは、このような点の数はどうやって求めれば良いのでしょうか。まず、全体の領域は下図の領域 (1), (2), (3) を使って、 $(1) - (2) - (3)$ という形で表すことができます。また、領域 (1) にある点の数は、 $R_i - L_i$ を小さい順にソートして二分探索することで簡単に計算できます。



問題は領域 (2) にある点の数ですが、下図のように横軸を $R_i - L_i$, 縦軸を R_i とした座標系に変換すると、長方形領域内に何個の点が含まれるかを求める問題に帰着されます。したがって、小課題 8 のように平面走査 + セグメント木を使うと、各シナリオにつき $O(\log N)$ で計算できます。領域 (3) も同様です。

以上の考察を使うと、各シナリオにつき計算量 $O(\log N)$ で感染者数を計算することができます。これでようやく、最後の小課題を解くことができました。



総評

この問題はかなり多くの考察ステップを必要とする難易度の高い問題です。これを解かれた方はヨーロッパ女子情報オリンピック (EGOI) で優勝を狙える実力がありますので、ぜひ自信を持っていただければと思います。